

■ Lab 13 — Workspaces

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Créé des **workspaces** `dev` et `prod`
- Déployé l'infrastructure sur chaque environnement
- Compris où Terraform stocke le **state file** par workspace
- Utilisé `terraform.workspace` dans le code

■ Étape 1 — Créer les fichiers

```
cd labs/lab13-workspaces
```

``backend.tf``

```
terraform {  
  backend "s3" {  
    bucket      = "tf-training-<votre-prenom>-982908300187"  
    key         = "terraform.tfstate"  
    region      = "eu-west-1"  
    use_lockfile = true  
    encrypt     = true  
  }  
}
```

■ ■ Remplacez `` par votre prénom. Ex : ``tf-training-alice-982908300187``

``versions.tf``

```
terraform {  
  required_version = ">= 1.15.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

``provider.tf``

```
provider "aws" {  
  region = var.region  
}
```

`variables.tf`

```
variable "username" {  
  description = "Votre prenom"  
  type        = string  
}  
  
variable "region" {  
  description = "Region AWS"  
  type        = string  
  default     = "eu-west-1"  
}
```

`locals.tf`

```
locals {  
  # terraform.workspace retourne le nom du workspace courant  
  environment = terraform.workspace  
  
  prefix = "lab13-${var.username}-${local.environment}"  
  
  # Adapter l'instance selon le workspace  
  instance_type = local.environment == "prod" ? "t2.small" : "t2.micro"  
  
  common_tags = {  
    Lab           = "lab13"  
    Username      = var.username  
    Environment   = local.environment  
    Workspace     = terraform.workspace  
    ManagedBy    = "Terraform"  
  }  
}
```

`main.tf`

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "lab13_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group Lab 13 - ${var.username} - ${local.environment}"

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab13_instance" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = local.instance_type
  vpc_security_group_ids = [aws_security_group.lab13_sg.id]

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance"
  })
}
```

`outputs.tf`

```
output "workspace" {
  description = "Workspace courant"
  value       = terraform.workspace
}

output "instance_id" {
  description = "ID de l'instance"
  value       = aws_instance.lab13_instance.id
}

output "instance_type_used" {
  description = "Type d'instance utilisé"
  value       = local.instance_type
}
```

■ Étape 2 — Créer les workspaces

```
terraform init

# Lister les workspaces (default existe toujours)
terraform workspace list

# Créer et basculer sur le workspace dev
terraform workspace new dev

# Créer le workspace prod
terraform workspace new prod

# Lister les workspaces
terraform workspace list
```

■ Étape 3 — Déployer en dev

```
# Basculer sur dev
terraform workspace select dev

# Vérifier le workspace courant
terraform workspace show

# Déployer
terraform apply -var="username=<votre-prenom>"
terraform output workspace
terraform output instance_type_used
```

Résultat attendu :

```
workspace          = "dev"
instance_type_used = "t2.micro"
```

■ Étape 4 — Déployer en prod

```
terraform workspace select prod
terraform apply -var="username=<votre-prenom>"
terraform output workspace
terraform output instance_type_used
```

Résultat attendu :

```
workspace          = "prod"
instance_type_used = "t2.small"
```

■ Étape 5 — Observer les state files dans S3

```
# Les workspaces créent des dossiers séparés dans S3
aws s3 ls s3://tf-training-<votre-prenom>-982908300187/env:/
```

Résultat attendu :

```
env:/dev/terraform.tfstate
env:/prod/terraform.tfstate
```

■ Terraform stocke les states des workspaces non-default dans `env://terraform.tfstate`.

Le workspace `default` utilise `terraform.tfstate` à la racine.

■ Étape 6 — Nettoyage

```
# Détruire prod
terraform workspace select prod
terraform destroy -var="username=<votre-prenom>"

# Détruire dev
terraform workspace select dev
terraform destroy -var="username=<votre-prenom>"

# Revenir sur default et supprimer les workspaces
terraform workspace select default
terraform workspace delete dev
terraform workspace delete prod
```

■ Validation du Lab

#	Critère	Validé
1	Workspaces `dev` et `prod` créés	■
2	`terraform.workspace` retourne le bon nom	■
3	`instance_type_used` = `t2.micro` en dev	■

#	Critère	Validé
4	<code>`instance_type_used` = `t2.small`</code> en prod	■
5	State files séparés dans S3 <code>`env:/dev/`</code> et <code>`env:/prod/`</code>	■
6	<code>`terraform destroy`</code> supprime les ressources sur chaque workspace	■

■ ****Fin du Lab 13 — Vous gérez plusieurs environnements avec les workspaces !****

*Le Lab 14 introduit ****terraform import**** pour gérer des ressources créées hors Terraform.*